ENEL 351 Project Report

Muhammad Hasaan Toor 200449166

April 11th, 2023

Project Tasks, Functions and Demonstrations

My project successfully achieved most of what I wanted it to. As a quick reminder, my robot consists of wheels, motors, and various sensors to move itself based on two programmed modes. The first mode is “Wall Following Mode” in which the robot uses an analog distance sensor to detect a wall to its left, following it. The second mode is “Line Avoidance Mode” in which the robot will use a digital line sensor to avoid any black lines that it drives over. The robot starts by default in Wall Following Mode and the blue USER switch on the microcontroller can be used to toggle between the two modes.

The actual functions of the robot slightly differ from those outlined in the Functional Specification Document, and I unfortunately could not get my indicator LED’s working without messing up something else. The reasons behind these shortcomings will be discussed later in the report.

I have also uploaded footage of my robot in both of its operating modes (and again, the answers to questions such as why it can only follow outside corner walls will be given further down below) (demos recorded in 2x speed):

Wall Following Mode: https://www.youtube.com/shorts/QLGb6LNNCp4
Line Avoidance Mode: https://www.youtube.com/shorts/xQRGO2_YNKI

Component List

- 1x Digital Line Sensor - QRE1113 Miniature Reflective Object Sensor
- 1x Analog Distance Sensor - Sharp GP2Y0A51SK0F Analog Distance Sensor - 2cm to 15cm
- 2x Motor Drivers - Dual Driver TB6612FNG v2
- 4x Motors - DC Gearbox Motor "TT Motor", 200RPM, 3 to 6VDC
- 1x Chassis and 4x Wheels - YIKESHU 4WD 2 Layer Smart Robot Car Chassis Kit with Battery Box
- 1x Microcontroller - Nucleo-64 STM32F103RB
- 4x 1.5V Batteries - AA batteries daisy chained together for 6V
- 1x 9V Battery
- 1x Breadboard
- Several Male-to-Male, Male-to-Female and Female-to-Female wires

Connection Details

User Controls:

Blue USER Switch — Toggle between wall following and line avoidance mode

I/O Details:

Battery Box + 1.5V Batteries — provides power/voltage to the 2 motor drivers controlling the motors and wheels.

9V Battery — provides power/voltage to the STM32F103RB board, as a USB connection is impossible while on the move.

Motor Drivers — input is connected to the F103RB's internal pulse width modulation (PWM) via several GPIO ports, utilizing 0-3.3V PWM to control the Duty Cycle of the 6V motors via output. The motors are connected to the wheels, therefore allowing me to control the speed of the motors/wheels and allowing movement of the robot in general.

Line Sensor — output is connected to a GPIO port in order to provide a digital input to the F103RB board, this digital input is used in the code to avoid the lines it detects.
Distance Sensor — output is connected to the F103RB's internal analog to digital converter (ADC) as this sensor provides an analog input. The analog input will be converted via the ADC to maintain a certain distance from a nearby wall to its side.

Program Listings

Used only Keil uVision as the project's development environment, using C code to program the robot. There is a main function and a util library, which contains all the necessary functions for initializing the STM32F103RB to the required specifications. For example it initializes the system clock, the 4 channels of TIM3 all of which are in PWM mode, the ADC and GPIO ports. Functions were made to control the PWMs, ultimately allowing for control of the speed of each of the 4 motors using the motor drivers. The inputs from the sensors were used to determine when to turn left, when to turn right, and when to keep straight. Here is a snippet of code from my GPIO initialization.

```
// Set the config and mode bits for:
// PA6, PA7, PB0, PB1 (PWM outputs -> CNF MODE = 1011 -> AF push pull output)
// PA1, PA4 (Logic 1's or 0's -> CNF MODE = 0011 -> push pull output)
// PA5 (Green LED -> CNF MODE = 0011 -> push pull output)
// PA0 (Distance Sensor analog input ADC -> CNF MODE = 0000 -> analog input)
// PB7 (Line Sensor digital input -> CNF MODE = 1000 -> input with pull up / pull down)
```

Limitations and Testings

I will go over some of the limitations of my robot and how it compares to the Functional Specification Document and then detail the process it took to get the final product. To begin, I had a discussion with Dave on one of the days where he was in the lab helping students with their projects. I told him my plan about wall following and line following mode, after some discussion I

learned that I could not do what I wanted to do with just a singular sensor for each mode.

For wall following mode, I am limited to only being able to follow walls on the left side of the robot, and only follow their turns if they make outside corner turns, as if there was an inside corner, my robot would simply run into the wall in front of it since it can only sense to it's left. So my algorithm is pretty simple, if the distance detected is closer than 4.5 cm, turn right as it is too close to the wall, if the distance is farther than 4.5 cm but closer than 7 cm, keep straight and finally if the distance is farther than 7 cm, turn left as it is too far away from the wall, this can happen either if the robot drifts too far right next to a straight wall or if a outside corner appears, as shown in my demonstration YouTube video. Overall, not too far of a deviation from my Func. Spec.

The line following mode was impossible with just one sensor, I would need at least 3 line sensors to detect whether to go straight, left, or right. I decided the next best option I could do with only one sensor is to do a line avoidance mode instead. This idea was also pretty simple, if a black surface/line is detected, then turn to the right, away from it, otherwise keep straight. I just arbitrarily chose to turn right as again, with only one sensor it would be impossible to determine whether to turn left or turn right, so only one direction can be chosen. This was a major deviation from the Func. Spec. plan but overall it still utilized the line sensor as intended, so I am satisfied with how it turned out.

Once I had figured out my new plan it was a simple matter of wiring everything together. I initially set up my distance sensor, and tested it on walls around my house. It worked mostly well, but would sometimes overturn when past a corner causing it to crash into the wall, it was very inconsistent.

After this I constructed my test course to conduct more consistent tests, and using trial and error I adjusted the distance thresholds for turning right, turning left and keeping straight and landed on 4.5 cm and 7 cm as they resulted in the most consistent results. I also messed around with the speeds of the motors when turning, in which case I found it best to turn the duty cycle of wheels to 0%, essentially locking them, when wanting to turn as anything higher would result in them carrying forward momentum and not actually going as slow as I wanted them to. This was probably the most difficult and tedious part of my project, the same exact code on the same exact course would give different results, as the quality of the sensors, motors and wheels is very low. So having to run tests, change some values and repeat got very annoying.

The distance sensor was a lot more straightforward, the only trouble I ran into with it is that I initially had it too far high up on my robot, as it only has a range of about 5 mm. Once that problem was fixed, it was detecting black tape on white surfaces, so it was just a matter of applying tape to my test course. I had to apply several layers as the robot would not be able to turn fast enough with only one layer.

The next big problem actually came after I had a successful working wall following and line avoidance mode. It happened when I tried adding three external LED's to indicate whether the robot was turning left, keeping straight, or turning right as outlined in my Func. Spec. Now theoretically this worked fine, I had wired the LED's up, set up new GPIO ports for them and tested everything while the STM32 was being powered via a USB connection (since I was loading the new code into its flash). I started the robot and used my hand as a "wall" to test out the LED's and it worked as expected, when turning left the left LED would come on, when going straight, the middle one

and when turning right the right one. So I switched from USB power to the 9V battery power and to my surprise things were not working as they should have been. The leftmost LED was flickering and the bottom right motor/wheel was not spinning no matter what I did. I found this very odd as I had not changed anything in terms of wires and it was just working on USB power. So I switched back to USB power and without changing any code, tested it out and it worked as intended again. I again tested with battery power and the same issues happened, so I am not sure what happens internally when the power supply is changed from USB to battery (to use battery power, you have to move a jumper on the STM32 as told to me by Dave) but it clearly changes some things and doesn't work functionally exactly like USB power mode. This was very frustrating and led to me scrapping anything to do with the LED's as I did not want to compromise my main two desired tasks.

Photos